MANUAL DE INGENIERÍA Y ARQUITECTURA

Guía Completa para la Creación de un Bot de WhatsApp

Arquitectura integral de producción basada en **WhatsApp Cloud API de Meta**, backend reactivo con **Node.js y Express**, base de datos **MySQL** y panel de administración en **React**.

📱 Meta Cloud API v21.0 ⚡ Webhooks Seguros 🗄️ MySQL & Emojis utf8mb4

🔄 Sesiones y Máquina de Estados ⚙️ CRUD de Mensajes Dinámicos 🏥 Turnos Médicos

PROYECTO	AUTOR / RESPONSABLE	FECHA DE EDICIÓN	VERSIÓN
OscarSoft Turnos	Oscar Stella	Octubre 2026	1.0 - Producción

1. Introducción y Arquitectura General

El presente documento recopila exhaustivamente la metodología técnica, diseño de arquitectura y código fuente implementado para construir el **Bot Oficial de WhatsApp** integrado al sistema de gestión de turnos médicos **OscarSoft Turnos**.

> 💡 **¿Por qué WhatsApp Cloud API Oficial de Meta y no librerías no oficiales?**
>
> A diferencia de librerías basadas en emulación de navegador web (como Baileys, WPPConnect o Puppeteer), la **API oficial en la nube de Meta** garantiza:
>
> - Cero riesgo de baneo o bloqueo del número corporativo.
> - Alta disponibilidad garantizada con SLA oficial de Meta.
> - Entrega inmediata de webhooks mediante protocolo HTTP POST seguro.
> - Soporte de botones interactivos, listas, plantillas de marketing y comprobantes verificados.

Diagrama de Flujo de la Arquitectura

La interacción del usuario desde WhatsApp viaja a través de la infraestructura distribuida de la siguiente manera:

```
[ Paciente en WhatsApp ]
          |
          ▼  (Mensaje de texto / opción seleccionada)
[ Servidores de Meta / WhatsApp Cloud API ]
          |
          ▼  (Webhook HTTP POST /api/turnos/whatsapp/webhook)
[ Servidor Node.js / Express (Backend) ]
          |
          ├──► [ userSessions (Memoria RAM) ] : Rastreo de estado conversacional
          |
          ├──► [ MySQL: bot_respuestas ] : Textos, plantillas dinámicas y FAQs
          |
          ├──► [ MySQL: usuarios / turnos / agendas ] : Lógica médica en tiempo real
          |
          ▼  (Llamada HTTP POST a https://graph.facebook.com/v21.0/{id}/messages)
[ Servidores de Meta ]
          |
          ▼  (Respuesta inmediata en el chat)
[ Paciente en WhatsApp ]
```

Componentes Clave del Sistema

Componente	Tecnología / Ruta	Responsabilidad
Meta Cloud API	Graph API v21.0	Recepción del mensaje móvil y envío del mensaje saliente.
Webhook Router	`server/routes/whatsapp.js`	Verificación GET con Meta y procesamiento POST de eventos.
Motor de Sesiones	`userSessions (Map)`	Control de la conversación interactiva paso a paso por teléfono.
Repositorio de Respuestas	Tabla `bot_respuestas`	Textos parametrizables, sin mensajes cableados en código.
Panel Administrativo	React (`DashboardPage.jsx`)	CRUD web para crear y editar respuestas y disparadores del bot.

2. Configuración en Meta for Developers

Para operar con la API Oficial de WhatsApp Cloud se deben seguir los siguientes pasos en la consola de Meta:

1. **Creación de la Cuenta y Aplicación**
   Ingresar en developers.facebook.com con la cuenta comercial de Facebook. Ir a **Mis Apps > Crear App** y seleccionar el tipo de caso de uso **Otro > Negocios (Business)**.

2. **Agregar el Producto WhatsApp**
   En el panel lateral de la aplicación creada, buscar **WhatsApp** y presionar **Configurar**. Esto habilitará la consola de WhatsApp Cloud API.

3. **Extracción de Credenciales Críticas**
   En la sección **WhatsApp > Primeros Pasos (API Setup)**, se generan los 3 identificadores obligatorios que deben almacenarse en el archivo `.env`:

```
# Identificadores de Meta for Developers
APP_ID=1783214182577627
WHATSAPP_PHONE_NUMBER_ID=1354333314430195
WHATSAPP_WABA_ID=1990059401689850

# Token Permanente generado en Meta Business Manager
WHATSAPP_ACCESS_TOKEN=EAAZAV0ufpNds...

# Token secreto que inventamos para verificar el Webhook
WHATSAPP_VERIFY_TOKEN=oscarsoft_turnos_wa_token_2026
```

> ⚠️ **Token Temporal vs. Token Permanente de Producción**
> El token inicial que provee la consola de Meta expira a las **24 horas**. Para producción es mandatorio crear un **Usuario del Sistema (System User)** en business.facebook.com con rol de Empleado o Administrador, asignarle la aplicación de WhatsApp y generar un token sin fecha de expiración con los permisos: `whatsapp_business_messaging` y `whatsapp_business_management`.

4. **Configuración del Webhook en Meta**
   En la solapa **Configuración de WhatsApp > Webhook**:
   - **URL de devolución de llamada:** `https://tu-dominio.com/api/turnos/whatsapp/webhook`
   - **Identificador de verificación (Verify Token):** Debe coincidir exactamente con `WHATSAPP_VERIFY_TOKEN` configurado en tu servidor.
   - **Campos de suscripción:** Suscribir obligatoriamente el campo `messages`.

3. Base de Datos: Cero Hardcodeo y Emojis

Una de las reglas arquitectónicas más importantes del sistema es que **ningún mensaje fijo ni flujo conversacional debe quedar cableado en el código JavaScript**. Toda respuesta proviene de la tabla `bot_respuestas`.

> ✅ **Soporte Mandatorio de Emojis: utf8mb4**
>
> Los mensajes de WhatsApp utilizan activamente emoticones (🏥, 🗓️, ⏰, 👨‍⚕️, ❌, ✅). En MySQL, el juego de caracteres estándar `utf8` solo soporta hasta 3 bytes, lo que causa un error fatal al intentar guardar un emoji. **Tanto la base de datos como las tablas y la conexión de Node.js deben crearse en `utf8mb4` con intercalación `utf8mb4_unicode_ci`**.

Estructura DDL de la Tabla `bot_respuestas`

```
CREATE TABLE IF NOT EXISTS bot_respuestas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  clave VARCHAR(80) NOT NULL UNIQUE,
  titulo VARCHAR(150) NOT NULL,
  descripcion VARCHAR(255) NULL,
  disparador VARCHAR(200) NULL,
  mensaje TEXT NOT NULL,
  activo TINYINT(1) DEFAULT 1,
  creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  actualizado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Variables Dinámicas del Sistema

Las plantillas de mensaje en la base de datos contienen marcas de sustitución entre llaves `{variable}` que el motor del backend reemplaza en tiempo de ejecución:

Variable	Descripción	Ejemplo de Reemplazo
`{medico_nombre}`	Nombre y apellido completo del profesional	*Dr. Carlos Bocian*
`{paciente_nombre}`	Nombre del paciente identificado	*Oscar Stella*
`{fecha}`	Fecha del turno en formato argentino	*24/10/2026*
`{hora}`	Hora de la cita médica (formato 24hs)	*09:30*
`{sede}`	Nombre y dirección de la unidad de atención	*Sede Pasaje Gutiérrez (Pje. Gutiérrez 980)*
`{turno_id}`	Identificador único del comprobante	*#482*
`{cobertura}`	Obra Social y Plan del paciente	*OSDE - Plan 210*

Mecanismo de Lectura y Fallback en JavaScript

```
async function getBotMessage(clave, fallback, variables = {}) {
  try {
    const [rows] = await pool.query(
      'SELECT mensaje FROM bot_respuestas WHERE clave = ? AND activo = 1 LIMIT 1',
      [clave]
    );
    let text = (rows.length > 0 && rows[0].mensaje) ? rows[0].mensaje : fallback;
    for (const [k, v] of Object.entries(variables)) {
      text = text.replaceAll(`{${k}}`, v || '');
    }
    return text;
  } catch (err) {
    let text = fallback;
    for (const [k, v] of Object.entries(variables)) {
      text = text.replaceAll(`{${k}}`, v || '');
    }
    return text;
  }
}
```

4. Implementación del Backend (Node.js & Express)

El backend gestiona dos responsabilidades esenciales: validar el webhook y recibir los mensajes entrantes.

1. Verificación del Webhook por Meta (Endpoint GET)

Cuando se configura el Webhook en el portal de Meta, sus servidores disparan una consulta GET con tres parámetros:

- `hub.mode`: Debe tener el valor `subscribe`.
- `hub.verify_token`: Debe coincidir exactamente con nuestro `WHATSAPP_VERIFY_TOKEN`.
- `hub.challenge`: Un número aleatorio que nuestro servidor debe devolver con código HTTP 200.

```
router.get('/webhook', (req, res) => {
  const mode = req.query['hub.mode'];
  const token = req.query['hub.verify_token'];
  const challenge = req.query['hub.challenge'];
  const expectedToken = process.env.WHATSAPP_VERIFY_TOKEN;

  if (mode && token) {
    if (mode === 'subscribe' && token === expectedToken) {
      console.log('✅ Webhook verificado exitosamente por Meta');
      return res.status(200).send(challenge);
    }
    return res.sendStatus(403);
  }
  return res.sendStatus(400);
});
```

2. Recepción de Mensajes del Paciente (Endpoint POST)

> ⚡ **Regla de Oro de Meta: Responder 200 OK Inmediatamente**
>
> Meta exige que el endpoint POST responda con código HTTP 200 en **menos de 3 segundos**. Si el servidor demora consultando la base de datos antes de enviar la respuesta HTTP, Meta considera que hubo un fallo y reintenta enviar el mismo mensaje una y otra vez en bucle. **Siempre debe ejecutarse `res.status(200).send('EVENT_RECEIVED')` antes o al inicio del procesamiento.**

3. Normalización Telefónica para Argentina (Prefijo 9)

En Argentina existe una particularidad técnica con WhatsApp: los teléfonos móviles a veces son reportados por Meta con el prefijo `549...` (13 dígitos) y a veces como `54...` (12 dígitos). Para evitar que los mensajes fallen, se implementó un mecanismo de reintento dual:

```
async function sendWhatsAppMessage(toPhone, messageText) {
  const phoneNumberId = process.env.WHATSAPP_PHONE_NUMBER_ID;
  const token = process.env.WHATSAPP_ACCESS_TOKEN;

  let targetPhones = [toPhone];
  const digits = String(toPhone).replace(/\D/g, '');
  if (digits.startsWith('549') && digits.length === 13) {
    targetPhones = ['54' + digits.slice(3), toPhone];
  } else if (digits.startsWith('54') && digits.length === 12) {
    targetPhones = [digits, '549' + digits.slice(2)];
  }

  const url = `https://graph.facebook.com/v21.0/${phoneNumberId}/messages`;

  for (const phone of targetPhones) {
    const res = await fetch(url, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${token}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        messaging_product: 'whatsapp',
        to: phone,
        type: 'text',
        text: { preview_url: false, body: messageText }
      })
    });
    if (res.ok) return true;
  }
  return false;
}
```

5. Máquina de Estados y Flujos Conversacionales

Cada usuario mantiene una sesión en memoria vinculada a su número telefónico (`userSessions.get(phone)`).

Estado (Step)	Objetivo	Siguiente Paso
`INIT`	Menú de bienvenida o detección de FAQ por palabra clave.	Según número elegido (1 a 5)
`AWAIT_MEDICO_SEARCH`	El usuario escribe el nombre o apellido del médico.	`SELECT_MEDICO`
`SELECT_MEDICO`	Elige el médico de la lista coincidente (1, 2, ...).	`SELECT_FECHA`
`SELECT_FECHA`	Elige una de las próximas 5 fechas disponibles.	`SELECT_HORA`
`SELECT_HORA`	Elige uno de los horarios libres (09:00, 09:30, ...).	`AWAIT_DNI`
`AWAIT_DNI`	Ingresa DNI del paciente para validar titularidad.	`SELECT_COBERTURA` o Invitación
`CONFIRM_CANCELACION`	Confirmación explícita (1: Sí / 2: No) antes de anular.	Liberación de turno en DB

Flujo 1: Agendamiento y Validación de Paciente

Si el paciente ingresa un DNI que **no existe** en la base de datos de usuarios, el bot no se rompe ni cancela abruptamente; responde con la plantilla `dni_no_registrado`, ofreciéndole el link web oficial para completar sus datos personales y registrar su carnet de obra social:

> **Mensaje enviado por el bot:**
> *"❌ DNI no registrado. No encontramos ninguna cuenta registrada con el DNI 32123456. Para poder agendar tu turno, primero registrate en: https://oscarsoft.com.ar/turnos/login?registro=1"*

Flujo 2: Consulta y Cancelación en 2 Pasos

Para cancelar un turno agendado (Opción 5), el sistema exige:

1. Ingreso del DNI para listar sus turnos activos futuros.
2. Selección del turno específico mediante su número de lista.
3. **Confirmación previa de seguridad:** El bot pregunta explícitamente:
   `*1* - ✅ Sí, cancelar este turno` / `*2* - ❌ No, conservar mi turno`.
   Solo si presiona 1 se ejecuta la sentencia SQL de liberación del turno y se notifica al paciente.

Disparadores de FAQ Inteligentes

La tabla `bot_respuestas` posee la columna `disparador`. Si un paciente escribe en cualquier momento palabras como *"sedes"*, *"donde atienden"*, *"requisitos"* o *"carnet"*, el bot salta directamente a la respuesta adecuada sin necesidad de navegar el menú numérico.

6. Panel de Administración Web (Dashboard React)

Para empoderar al equipo médico y administrativo sin depender de desarrolladores para cambiar textos, se construyó una interfaz interactiva de administración.

Funcionalidades del Panel Administrativo

- **Listado de Respuestas Activas:** Muestra cada plantilla, su clave de sistema, título descriptivo, disparadores configurados y estado de activación.
- **Editor en Vivo:** Modal con área de texto enriquecida que permite modificar los mensajes, añadir saltos de línea y emojis.
- **Guía de Variables:** El panel indica qué variables están permitidas para cada plantilla (ej: `{medico_nombre}`, `{fecha}`, etc.).
- **Activación / Desactivación:** Switch para deshabilitar una respuesta o disparador con un solo clic.

Rutas del Servidor para el CRUD (`server/routes/admin.js`)

```
// Obtener todas las respuestas y preguntas del bot
router.get('/bot-respuestas', authMiddleware, requireAdmin, async (req, res) => {
  const [rows] = await pool.query('SELECT * FROM bot_respuestas ORDER BY id ASC');
  res.json({ status: 'ok', data: rows });
});

// Actualizar una respuesta existente
router.put('/bot-respuestas/:id', authMiddleware, requireAdmin, async (req, res) => {
  const { id } = req.params;
  const { titulo, descripcion, disparador, mensaje, activo } = req.body;
  await pool.query(
    'UPDATE bot_respuestas SET titulo = ?, descripcion = ?, disparador = ?, mensaje = ?, activo = ?
WHERE id = ?',
    [titulo, descripcion, disparador, mensaje, activo ? 1 : 0, id]
  );
  res.json({ status: 'ok', message: 'Respuesta actualizada exitosamente.' });
});
```

> **Seguridad en la API de Administración**
>
> Todos los endpoints de administración exigen autenticación mediante tokens JWT y verifican que el rol del usuario sea `superadmin` o `admin` antes de permitir cualquier mutación de datos.

7. Guía de Despliegue, Testing y Buenas Prácticas

Testing en Entorno Local (Desarrollo)

Como los servidores de Meta necesitan enviar peticiones HTTP a una dirección accesible desde internet con certificado SSL (HTTPS), no se puede ingresar `http://localhost:5060` directamente en Meta.

1. **Levantar un Túnel HTTPS Seguro**
   Se puede utilizar **Cloudflare Tunnels** o **Ngrok**:

   ```
   ngrok http 5060
   ```

   Ngrok otorgará una URL temporal pública (ej: `https://abcd-123.ngrok-free.app`).

2. **Configurar la URL en Meta**
   Pegar la URL generada: `https://abcd-123.ngrok-free.app/api/turnos/whatsapp/webhook` y verificar con el token.

3. **Enviar Mensaje de Prueba desde un Teléfono Autorizado**
   En modo Desarrollo de Meta, solo los números registrados en la lista de *Destinatarios de Prueba (To Phones)* pueden enviar y recibir mensajes.

Despliegue en Producción

Checklist de Salida a Producción:

1. **Nginx Reverse Proxy:** Configurar Nginx para redirigir el tráfico del subdominio o ruta `/api/turnos/whatsapp` hacia el puerto de Node.js (5060).
2. **Certificado SSL/TLS Válido:** Meta rechaza de inmediato cualquier webhook con certificado autofirmado o vencido. Usar Certbot (Let's Encrypt).
3. **Gestor de Procesos PM2:** Ejecutar la aplicación con `pm2 start server/index.js --name "turnos-backend"` con reinicio automático.
4. **Verificación de Negocio en Meta:** Para enviar mensajes a cualquier paciente sin restricción de prueba, se debe completar la *Verificación de Empresa (Business Verification)* en el Administrador Comercial de Meta.

Errores Frecuentes y Cómo Solucionarlos

Error / Síntoma	Causa Raíz	Solución Definitiva
`Verification failed (403)`	El `verify_token` ingresado en Meta no coincide con el `.env`.	Verificar mayúsculas y espacios en `WHATSAPP_VERIFY_TOKEN`.
`Error 190 (Invalid OAuth)`	El token de acceso de 24 horas expiró.	Crear System User permanente en Business Manager y renovar `WHATSAPP_ACCESS_TOKEN`.
`Incorrect string value (emojis)`	La tabla de MySQL está en `latin1` o `utf8` (3 bytes).	Ejecutar `ALTER TABLE bot_respuestas CONVERT TO CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;`
Bucle infinito de mensajes de Meta	El endpoint POST tardó más de 3 segundos en responder.	Enviar `res.status(200).send('EVENT_RECEIVED')` al inicio de la función.

Documento generado automáticamente para el proyecto OscarSoft Turnos • Todos los derechos reservados.